

COP 290 Assignment 2

Task Board

Ojas Pednekar – 2024CS10595
Mridul Chhipa – 2024CS10411

March 22, 2026

[Github Repository Link](#)

1 Introduction

This project implements a Task Board, a project management and issue-tracking application inspired by tools such as Jira. It allows teams to collaboratively manage tasks using Kanban boards, workflows, and role-based access control.

The system supports task creation, assignment, workflow transitions, notifications, and collaboration through comments and mentions. The goal of this project is to provide a scalable and structured platform for tracking work while maintaining traceability and proper access control.

2 Tools and Libraries Used

- **TypeScript:** The base language used for most of the application.
- **CSS:** Used for styling and positioning of UI elements.
- **Prettier and ESLint:** Used for enforcing a consistent coding style.
- **React:** Used for building the frontend using JSX and component-based architecture.
- **Vite:** Provides fast builds and hot module replacement for development.
- **React Router:** Enables client-side routing.
- **Express.js:** Handles backend API requests and routing.
- **JWT:** Used for secure authentication and session management.
- **PostgreSQL:** Database used for persistent storage.
- **Prisma:** ORM used for communication between backend and database.
- **WebSockets:** Used for real-time notification delivery.

2.1 Architecture Working

At startup, HTTP servers are launched for both the frontend and backend. A WebSocket server is attached to the backend HTTP server, and the database is initialized.

The user interacts with the frontend, which is built using React. These interactions trigger API calls using asynchronous `fetch` requests. These requests are handled by backend routes implemented in Express.

The backend processes the requests using TypeScript code and interacts with the PostgreSQL database through Prisma. Any changes to application data are reflected in the database.

Additionally, WebSockets are used for sending and receiving real-time notifications between the frontend and backend.

3 Role-Based Access Control

- **Global Admins:** A Global Administrator is seeded into the system. This administrator can promote other users to Global Admins. However, administrators cannot be demoted back to regular users.
- **Global Admin Privileges:** A Global Admin can create new projects, add users to projects, assign roles, and remove users from projects. Within a project, however, a Global Admin is treated as a Project Viewer unless they created the project, in which case they are the Project Admin.
- **Project Admins:** Project Admins are assigned by Global Admins. They can create boards, manage workflows, define task transitions, and modify project settings. They can also add or remove users and assign roles within the project. Workflow order can only be changed when the workflow is empty.
- **Project Members:** Project Members have limited access to project and board management. They can create, edit, delete, and update tasks, but cannot modify workflows or project-level settings.
- **Project Viewers:** Project Viewers have read-only access to boards and tasks. They can also view threads and comments of each task. If there are multiple global admins and there is a global admin who has not created a project then that user is a project viewer of this project

4 Design Decisions

- **User Settings:** Users can edit their name through the account page. They can also provide a URL to set an avatar. Email addresses are unique and cannot be changed after registration.
- **Parent-Child Task Relationships:** Each Story (parent task) includes a copy button. When creating or editing a task, the copied ID can be used to assign it as a child of the Story. Children can be viewed using a dedicated button on the Story.
- **Story Status Updates:** Stories are displayed alongside normal tasks. If a Story has no children, its status can be updated manually. If it has children, its status is derived from the first child task based on workflow order. Changes in child tasks affect the parent Story's position.
- **Task Transitions:** Task transitions are strictly controlled. Project Admins define valid transitions for workflows. Invalid transitions are blocked.
- **Notifications:** Notifications are triggered for task assignments, status updates, edits, thread creation, comments, replies, and mentions.
- **Mentions:** Mentions are triggered using the format `@email@example.com`. The email is parsed, and a notification is sent to the corresponding user.
- **Error Handling:** Invalid operations (such as incorrect transitions or assigning users outside the project) are blocked by both frontend and backend guards. The frontend prevents invalid API calls, while backend middleware ensures security. Errors are communicated through popup messages to improve user experience.
- **Responsive Design:** The application is designed to be responsive across devices. CSS modules adjust layout dynamically based on screen size. For example, projects are displayed in a horizontally scrollable container on smaller screens.
- **Authentication and Authorization:** Logging in creates a session stored in the database. Access and refresh tokens are stored in HTTP-only cookies. Refresh tokens are used to renew sessions, and are invalidated upon logout. Access tokens expire in 15 minutes, while refresh tokens expire in 7 days. A new refresh token is generated upon session renewal.
- **Context Management:** User and project data are stored using React Context and managed using reducers. This ensures efficient global state management.
- **Drag and Drop:** Drag-and-drop functionality is implemented for both tasks and columns. Task movement updates task status and parent Story state. Column movement reorders workflows.
- **WIP Limits:** Default WIP limits are applied to workflows. A limit of 0 or -1 represents no restriction. Custom limits can be set when creating new workflows.

- **Comments Refresh Strategy:** Instead of using reducers, the comments page is refreshed using a refresh key mechanism that triggers re-rendering via the `useEffect` hook.
- **User Data Security:** No API allows direct access to another user's data. All user data access is managed through database relations, ensuring scalability and security.
- **Audit Logs:** All backend API actions are logged in the database. A subset of these logs is displayed in the task activity section. This ensures traceability and helps in debugging.
- **Database Design:** The database schema is designed using relational models with proper constraints and relationships. An ER diagram is included separately.
- **API Design:** API routes follow REST principles. POST is used for creation, GET for retrieval, PATCH for updates, and DELETE for removal.

Structure: Routes are defined in `.routes.ts`, which call controller functions in `.controller.ts`. These controllers invoke service functions in `.services.ts`, which interact with the database. Middleware is used for authentication and authorization.

Status Codes: All API responses use appropriate HTTP status codes (2xx for success, 4xx/5xx for errors).

5 Database Design

Entity-Relationship Diagram

The following Entity-Relationship (ER) diagram outlines the core data models, their attributes, and the relationships that power the application's collaboration and workflow features.

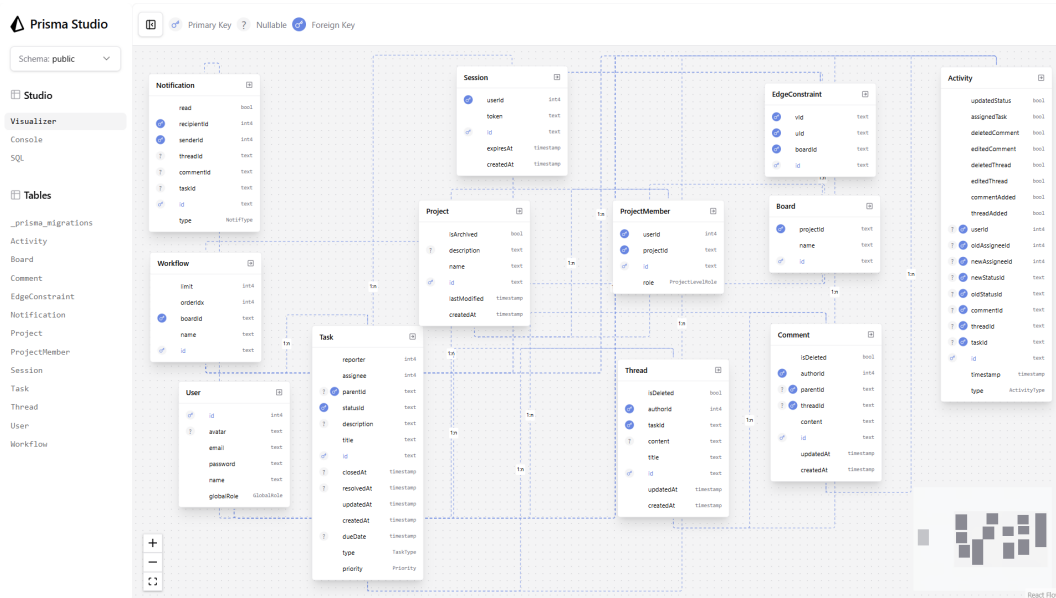


Figure 1: Physical ER Diagram mapping the system's database schema.

Core Tables

The tables are as Users, Projects, Tasks and Notifications

Column	Type	Constraint	Description
id	int4	PRIMARY KEY	Unique identifier for the user.
email	text	UNIQUE, NOT NULL	User's email address.
name	text	NOT NULL	User's full name.

Column	Type	Constraint	Description
id	text	PRIMARY KEY	Unique identifier for the project.
name	text	NOT NULL	The title of the project.
description	text	NULLABLE	Optional details about the project scope.
isArchived	bool	NOT NULL	Flag indicating if the project is archived.
createdAt	timestamp	NOT NULL	Timestamp of creation.
lastModified	timestamp	NOT NULL	Timestamp of the last update.

Column	Type	Constraint	Description
id	text	PRIMARY KEY	Unique identifier for the task.
projectId	text	FOREIGN KEY	References Projects(id) .
reporter	int4	FOREIGN KEY	References Users(id) who created it.
assignee	int4	FOREIGN KEY, NULLABLE	References Users(id) assigned to work on it.

Relationships

Based on the physical schema, the primary relationships are defined as follows:

- **Project to Tasks (1:N):** A single project can contain multiple tasks. This is enforced by a foreign key (**projectId**) within the **Task** entity.
- **User to Projects (M:N):** Handled via the **ProjectMember** junction table (shown in the ER diagram), allowing a user to join multiple projects and projects to have multiple users with specific roles.
- **User to Tasks (1:N):** A user can be associated with multiple tasks, either as the **reporter** or the **assignee**.
- **Task/Thread/Comment to Notifications (1:N):** Actions on various entities trigger notifications. The Notification table uses nullable foreign keys (**taskId**, **threadId**, **commentId**) to link the alert to its specific source entity.

Constraints

- **Entity Integrity:** Every table relies on a strictly defined Primary Key (**id**) to ensure unique record identification.
- **Referential Integrity:** Foreign keys strictly map to existing records (e.g., a **Task** cannot be assigned to an **assignee** ID that does not exist in the **Users** table).
- **Nullability constraints:** Indicated by the ? icon in the schema, fields like project **description** and notification context links (**taskId**, **commentId**) are explicitly allowed to be NULL to accommodate varying states of data completeness.

Indexing Strategy

To ensure the database remains performant as it scales, the following indexing strategies are implemented:

- **Primary Indexes:** B-Tree indexes are automatically created on all Primary Keys (e.g., **id** fields in **Projects**, **Tasks**, and **Notifications**).
- **Foreign Key Indexes:** Indexes are applied to highly queried foreign keys, specifically **projectId** in **Tasks**, and **recipientId** in **Notifications**, to optimize join operations and dashboard data retrieval.
- **Boolean/State Indexes:** Partial indexes can be applied to fields like **isArchived** in **Projects** or **read** in **Notifications** to quickly filter out active workflows or unread alerts without scanning the entire table.

Column	Type	Constraint	Description
id	text	PRIMARY KEY	Unique identifier for the notification.
type	NotifType	NOT NULL	Enum representing the kind of notification.
read	bool	NOT NULL	Flag indicating if the user has seen it.
recipientId	int4	FOREIGN KEY	References Users(id) receiving it.
senderId	int4	FOREIGN KEY	References Users(id) who triggered it.
taskId	text	FOREIGN KEY, NULLABLE	References Tasks(id) if applicable.
threadId	text	FOREIGN KEY, NULLABLE	References Threads(id) if applicable.
commentId	text	FOREIGN KEY, NULLABLE	References Comments(id) if applicable.

6 Testing

6.1 What We Tested

We wrote unit tests for backend services (Auth, Project, Board, Task, Comment, Notification), covering both success paths and all defined error/edge outcomes. This includes validation failures, missing entities, permission-related checks, notification side-effects, and data-mapping behavior for DTOs.

6.2 Tools Used

We used the Node.js built-in `node:test` runner with `assert` for assertions. Tests run via the `tsx` test command in the backend workspace.